ORCA: Building a Fast, Independent Platform on a System You Do Not Own

Nathan Curtis
Independent
nathan@nathancurtis.me
ORCID 0009-0008-0350-5799
DOI 10.5281/zenodo.23493408

Abstract

Many organizations run their core business on a vendor's system: a database they can read, whose schema, release cycle and API belong to someone else. Everything else grows around it. Tools query it directly, keep private copies that drift, and carry the vendor's quirks into every one of them.

This paper describes ORCA (**O**perational store, **R**ead models, **C**hange capture, **A**nti-corruption layer), an architectural pattern for that situation: own a translated, live copy for reads, and send every write back through the vendor. ORCA is assembled from known ideas. What it adds is how they are held together when the source is someone else's, and an exact account of which guarantees survive. It closes with what one implementation learned in production.

Keywords software architecture, change data capture, anti-corruption layer, read models, CQRS, vendor systems

Reference Format:
Nathan Curtis. 2026. ORCA: Operational store, Read models, Change capture, Anti-corruption layer. Version 1.0.
https://doi.org/10.5281/zenodo.23493408

This work is licensed under Creative Commons Attribution 4.0. Use it, adapt it, build on it; credit Nathan Curtis and link to orcapattern.org.
Version 1.0, October 2026. Source: github.com/nathannncurtis/orca.

1 The problem

The source here is a vendor's database: a practice-management, ERP or line-of-business product the business runs on. You can read it; its schema, release cycle and API belong to someone else. Everything built after it grows around it.

- **Direct reads.** Dozens of internal tools query the vendor's database, each with its own polling loop.
- **Private copies.** Each tool keeps its own partial copy; each drifts a little differently.
- **Leaking quirks.** The vendor's model (packed flag bytes, coded columns that need lookups, one table doing several jobs) leaks into every one of them.
- **A slow, incomplete API.** Too slow and too incomplete to build on, and changing it isn't an option.

This paper makes three contributions:

1. A way to keep a translated, live copy of a system you can read but not change, built on capture treated as a hint rather than a fact (3.1, 3.2).
2. A write path through the vendor's API that never resends blind, and reports one of three outcomes instead of claiming exactly-once delivery (4.3).
3. An exact statement of what survives: the invariants each part keeps, and the guarantees and non-guarantees of the whole (5).

It closes with what one implementation learned in production (10).

2 Why the usual answers fail

Four answers come up first; none of them fits.

Table 1. The usual answers, against what a platform on someone else's system needs.

Approach	The reads you need	Fast	Your own model	Vendor's schema untouched	In scope
Rebuild on the vendor's API *It lacks the reads you need.*	×	×	·	·	·
Replace the vendor *Out of scope.*	·	·	·	·	×
Turn on change tracking *That alters a schema you do not own.*	·	·	·	×	·
Mirror the database *You get speed, but every app still speaks the vendor's model.*	·	✓	×	·	·
ORCA *A translated, live copy for reads; every write back through the vendor.*	✓	✓	✓	✓	✓

✓ Met × Why it fails · Not the deciding issue

ORCA is the shape that remains: own a translated, live copy for reads, and send every write back through the vendor. It isn't a stack. Any language, database or message bus can implement it.

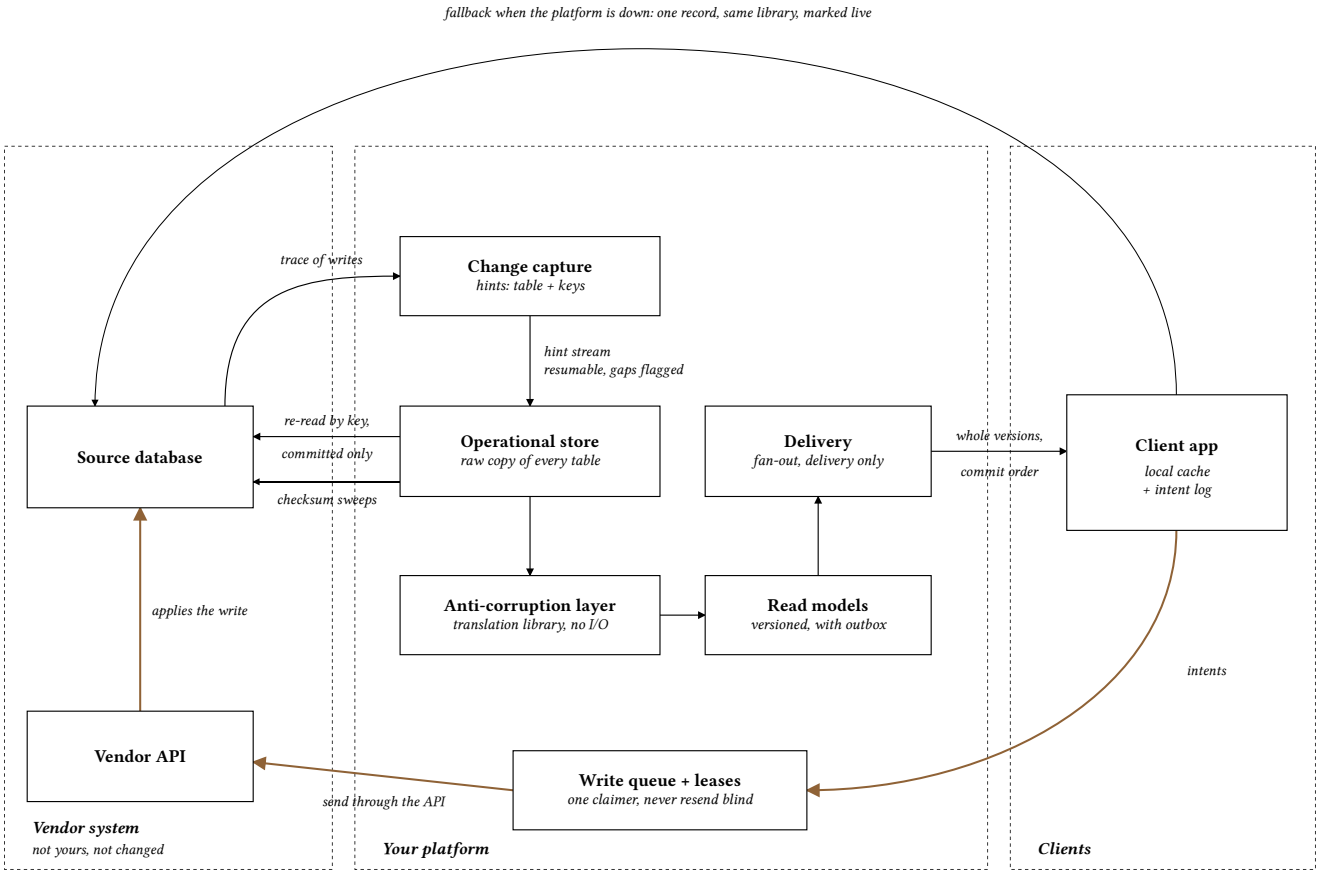


Figure 1. ORCA at a glance. Reads flow left to right and never touch the vendor except through re-reads, sweeps and a narrow fallback. Writes (in bronze) flow through the vendor's own API and are judged by what change capture brings back.

3 The pattern

3.1 Change capture

A low-latency, best-effort signal of which records changed, taken from outside the vendor's schema.

> *The obvious choice:* Turn on the database's own change tracking. *Why ORCA doesn't:* It alters a schema you don't own.

Capture tells the store where to look; nothing more. It's taken from outside the vendor's schema: a database trace session, a transaction-log reader, or network capture of the vendor's client traffic. Nothing in the vendor's schema is added or changed to produce it.

Capture is allowed to be wrong; it's never trusted for values. A record can arrive before its change commits, describe a change that rolls back, or not arrive at all. Everything downstream is built to survive.

Capture record. A table and (typically) the primary keys it touched; no values are carried.

Position. A point in the stream a consumer can resume from.

Gap. A flag that records after a position may have been lost.

Invariant C1 (Resumable). *A consumer reconnecting from a position receives the records after it, or a gap.*

Invariant C2 (Honest gaps). *If records may have been lost, the consumer is told; no gap means nothing was lost.*

These are the only guarantees capture gives. Reconciliation (3.2) covers what capture misses.

What a record says

> *The obvious choice:* Trust capture, as a change data capture pipeline usually does. *Why ORCA doesn't:* Capture taken from outside the schema is best-effort. Truth comes from a committed-only re-read.

A record names where a change happened; the store decides what to do with it. There are three shapes, with one response each:

- **A table and keys.** Re-read those rows (3.2).
- **A table only.** Reconcile the table.
- **Nothing, or a gap.** Reconcile everything the gap could cover.

Records naming tables the store doesn't copy (temporary tables and the like) are skipped.

Algorithm 1. Consuming capture

```
loop:
    batch, gap ← capture.read_after(position)
    if gap: request reconcile(all)
    for record in batch:
        if record.keys:  reread(record.table,
record.keys)
        elif record.table: request
reconcile(record.table)
    position ← batch.last
```

Where it's taken from

The database server is the one place every writer passes through. A trace session there sees the vendor's desktop clients, its API, its web portal and its scheduled jobs alike; capture taken from the API or from client traffic only sees the writers that use it.

The capturing service runs next to the database, writes to bounded rolling files, and serves records over a long poll with a resume position. A consumer that falls behind the files' retention gets a gap rather than a silent hole.

3.2 Operational store

Your own current and full copy of the vendor's data.

A copy of every vendor database table. Anything downstream reads from it; it converges on what the source has committed without needing any trust in what the capture is. Its four parts are: a paced first copy, a re-read on hint, reconciliation by key range, and a raw layer.

Hint. A capture record naming a table and (typically) primary keys; no values are carried.

Committed-only read. A read that sees only committed data; it waits on uncommitted writes rather than reading them through.

Range. A contiguous span of primary keys in one table.

Invariant S1 (Convergence). *Every committed source row eventually appears in the store with its committed values.*

Invariant S2 (No rolled-back residue). *A rolled-back value does not survive past the next re-read or reconciliation of its row.*

Invariant S3 (Ordered application). *An older source read is never written over a newer one.*

First copy

Each table is copied in primary-key order, in paced chunks; a chunk's rows and its last key commit together, making a restart resume after the last chunk is written. A failing chunk is retried at half the size, down to one row; the size grows back after each success. Capture runs alongside the copy, and the copy never overwrites a row capture has already written. A table counts as correct only after its first reconciliation.

Re-read on every hint

A hint names only rows; it never supplies values. The store rereads the named rows by primary key, with a committed-only read, and replaces its copy with the result.

Algorithm 2. Re-read on a hint

```
on hint(table, keys):
    if keys is empty:
        request reconcile(table)
        return
    rows ← source.read_committed(table, keys)
    with lock(table):
        store.replace(table, keys, rows)
```

A hint that arrives before its transaction ends will wait; the read returns the new values if it commits and the original values if it rolls back (S2). The capture position is saved after each batch is applied; a restart reapplies the last batch, and re-reads are idempotent.

The hint only decides where to look; the source decides the truth.

> *The obvious choice:* Trust the stream to keep the copy right. *Why ORCA doesn't:* Capture misses changes, such as trigger writes and unparsable statements. Only a comparison against the source is trustworthy.

Reconciliation

Both sides compute a row count and a checksum sum per range; only ranges that differ are re-copied, with a committed-only read.

Algorithm 3. Reconciling a table

```
reconcile(table):
    for range in key_ranges(table):
        if summary(source, range) ≠ summary(store,
range):
            rows ← source.read_committed(table, range)
            with lock(table):
                store.replace_range(table, range, rows)
```

It runs after every gap, after each table's first copy, and on a slow schedule; it always beats the copy and the stream. A change capture missed lies in a range whose summary differs, up to checksum collisions; the next pass copies it (S1).

A store row missing from a range is deleted only after a lookup by primary key confirms it's gone; two engines rarely order or compare text the same way, and a range boundary drawn by one can exclude a row in the other. A re-read and a reconciliation of the same table each hold a lock on it from source read to commit; an older read never lands after a newer one (S3). Large tables reconcile off-peak; small tables, and tables a hint named, at any time.

The raw layer

The vendor's tables, kept exactly with a checksum per row. Nothing downstream reads them; they let the translation layer (3.3) and the read models be rebuilt without touching the vendor. A table schema that changes is copied again.

3.3 Anti-corruption layer

A translation library that turns the vendor's model into yours.

> *The obvious choice:* Translate at the boundary, inside the service that reads the vendor. *Why ORCA doesn't:* Clients' fallback reads the

> source directly too. A library with no I/O, shared by both, gives the vendor's model one home.

The one place the vendor's model is understood. It takes rows and returns your shapes; it does no I/O. The store feeds it from the raw layer, and a client's fallback feeds it straight from the source (4.2). Both get the same answer.

Shape. A document in your model, built from one entity's rows: an order header, a notes log, a list row.

Lookup. A small vendor table that resolves a code to what it means.

Invariant A1 (One home). *Nothing outside the library reads the vendor's model.*

Invariant A2 (Pure). *A shape is a function of the rows it's given; the same rows always produce the same shape.*

What it translates

- **Packed values.** Flag bytes and coded columns, decoded into named fields.
- **Codes.** Resolved through lookups.
- **Tables doing several jobs.** Split by job.
- **Rows that always belong together.** Joined into one shape.

What it declares

Each shape lists the tables and columns it reads. That list decides which raw changes rebuild it (3.4) and what the fallback has to fetch.

Algorithm 4. Building a shape

```
build(entity, source):
    rows ← source.fetch(declared_tables, entity)   # store or vendor
    return translate(rows, lookups)
```

Checked against the source

A translation that's wrong is wrong everywhere at once, so it's checked. Every night, random entities are built from the store and compared field by field with the same entities built by independent queries straight from the source. A difference is read again a moment later and counts only if it persists.

3.4 Read models

Screen-shaped documents built by the translation layer from the raw layer.

What clients actually read. They're versioned, so a consumer always knows what it has and what it's missing.

Version. A number given to each committed change of read models; one version can cover several.

Outbox. The table recording each version's changes; the record every delivery reads from.

Dirty entity. An entity whose raw rows changed since its shapes were last built.

Invariant R1 (Commit order). *Version order equals commit order.*

Invariant R2 (Whole versions). *A consumer never holds part of a version.*

Invariant R3 (No skips). *A consumer reading after version v never misses a committed version.*

Rebuilding

A re-read or reconciliation that changes a raw row a shape depends on marks its entity dirty, in the same transaction. The shaper rebuilds dirty entities. A shape that comes out identical isn't written and takes no version. A changed lookup rebuilds everything that uses it.

Versioning

A plain sequence isn't enough; sequence values are handed out before commit, so a later commit can carry an earlier number. Each shaping transaction takes a single counter row, increments it, and holds its lock until commit; versions commit in order (R1). The same transaction writes one outbox row per changed shape.

Algorithm 5. Rebuilding an entity

```
rebuild(entity):
    begin
        v ← counter.increment()      # row lock held until commit
        for doc in build(entity, store):
            if doc ≠ stored(doc):
                write(doc, v)
                outbox.append(v, doc)
    commit
```

Reading

Consumers read the outbox by version, never by row, and a batch always ends on a version boundary. A consumer applies a whole version and only then saves it as its position (R2, R3). A shape that no longer exists keeps its row with an empty document, so its deletion is a change like any other.

4 Delivery, clients and writes

4.1 Delivery

A fan-out layer (a pub/sub cache, a websocket service) that carries committed versions from the outbox to everyone listening.

It's delivery only, never the record. Anything it loses is recovered from the outbox by position; losing a message costs latency, never correctness. It never splits a version (R2).

4.2 Clients

A client can keep its own embedded copy of the read models it needs (typically open records plus a recent window), fed by delivery and resumed by version. A client that reconnects after its position has expired resynchronizes rather than replaying.

Fallback

When the platform is down, a client may run the same translation library straight against the source; A2 makes the answer the same. The fallback is narrow: the one record on screen, rate-limited, and clearly marked as live. An outage of the platform must never become a flood of reads on the vendor.

4.3 Writes

Nothing writes to the vendor's database directly; every write goes through the vendor's own API.

> *The obvious choice:* Write to your own store, as CQRS assumes you own the write side. *Why ORCA doesn't:* You don't. Writes go through the vendor's API and are judged by what comes back.

Intent. A change the user asked for, recorded locally before it's sent.

Claim. One worker's exclusive ownership of an intent.

Outcome. The judgment on a sent intent: confirmed, state reached or undetermined.

Invariant W1 (One claimer). *Each intent has at most one claimer at a time, in order per record.*

Invariant W2 (Never resend blind). *An intent whose send is uncertain is resolved by observation, not by sending again.*

Invariant W3 (Evidence before done). *A write is reported done only with evidence; a write with side effects only when confirmed.*

Intents

A client records the user's intent in a durable local log and shows it at once; a short undo window is optional. A queue (e.g. `FOR UPDATE SKIP LOCKED`) gives each intent one claimer, in order per record (W1). That isn't exactly-once delivery. A worker can send and then crash before recording it.

> *The obvious choice:* Aim for exactly-once delivery, with idempotent retries. *Why ORCA doesn't:* A worker can send and then crash before recording it. ORCA replaces that with honesty: one claimer, never resend blind, and three outcomes.

Outcomes

A write is judged by what comes back through capture, not by the API's response alone.

- Confirmed Tied to this operation; for a create, the API returns the new record's ID and that exact record arrives.
- State reached The record holds the values sent, after the intent was sent, but nothing proves this request caused it. Enough for plain "set this field" changes.
- Undetermined The evidence can't tell. Never retried automatically; shown to the person.

Algorithm 6. Judging a write

```
judge(intent, observed):
    if intent.created_id and observed.id =
intent.created_id:
        return confirmed
    if observed holds intent.values and
observed.changed_after(intent.sent_at):
        return state_reached
    return undetermined
```

Anything with side effects (creating records, sending, billing) counts only when confirmed (W3).

Leases

Leases with fencing tokens coordinate your own writers. They're checked when an intent is queued and again before it's sent. They can't recall a request already in flight, and they mean nothing to people working directly in the vendor's application; conflicts with those people are detected afterwards through capture, not prevented. The vendor's own record locks, if visible, are mirrored in as a warning.

5 Guarantees

What the whole pattern promises, assembled from the invariants of its parts.

ORCA guarantees

- ✓ Every committed source change eventually appears in the store (S1).
- ✓ The store never keeps a rolled-back value past the next reread or reconciliation (S2).
- ✓ Version order equals commit order, and no consumer skips a committed version (R1, R3).
- ✓ A write is reported done only with evidence; a write with side effects only when confirmed (W3).
- ✓ The vendor's model never leaks past the translation layer (A1).

ORCA does not guarantee

- × That every change is seen through capture; capture is a hint (3.1).
- × That reads are as fresh as the source at every instant.
- × Exactly-once delivery of writes to the vendor (W2 replaces it).
- × That a write's cause can always be proved; undetermined exists.
- × Coordination with people using the vendor's application directly.

6 Failure cases

Each failure, what happens, and the invariant that holds through it.

Table 2. What happens when each part fails.

Situation	What happens	Holds
Capture misses a change (trigger writes, unparsable statements)	Reconciliation finds the differing range and re-copies it	S1
Capture reports a gap	The consumer requests reconciliation of everything the gap could cover	S1, C2
A change rolls back after its hint	The committed-only re-read returns the original values	S2
The first copy races a change	Each table counts as correct only after its first reconciliation	S1
A worker crashes after sending a write	The intent is resolved by observation; it is never resent blind	W2
A client reconnects after its position has expired	Full resync of its read models, not a replay	R3
The platform is down	Clients fall back narrowly to the source, visibly	A2
The vendor changes its schema	The raw layer shows it; the table is copied again and translations are rebuilt	S1, A1

7 A worked example

One order, followed through the system. "Order 42" is any record in the vendor's database. First the ordinary path, then the same order when things go wrong.

The ordinary path. State reached

The user sees the change at once, from their own intent log. The vendor's API applies it. Capture says only "orders, key 42". The store re-reads that one row with a committed-only read, rebuilds the read models for order 42 in one transaction that takes version 1001 and writes it to the outbox, and every client receives version 1001 as a whole. The intent compares what came back with what it sent: the values match and arrived after the intent was sent, so it is judged "state reached". For a create, the API's returned ID would make it "confirmed".

1. User: records the intent and shows it at once
2. User → Write queue: enqueues it; the lease is checked
3. Write queue → Vendor API: sends it; the lease is checked again
4. Vendor API → Source database: updates order 42
5. Source database → Change capture: statement completed
6. Change capture → Store: hint: orders, key 42
7. Store → Source database: re-reads order 42, committed only
8. Store: rebuilds order 42's read models as version 1001
9. Store → User: version 1001, delivered whole

Outcome: *state reached (a create would be confirmed by its ID)*

Capture loses the event.

Nothing arrives for order 42. The intent waits; after its timeout the store re-reads order 42 directly, and the next checksum sweep would find the range anyway. The intent then resolves on what it sees, never by sending again.

1. Write queue → Vendor API: sends the intent
2. Vendor API → Source database: updates order 42
3. Source database → Change capture: statement completed: the event is lost
4. User: timeout: nothing has come back
5. Store → Source database: re-reads order 42 for the intent
6. Store → User: changed: version 1001

Outcome: *resolved by observation, never resent*

The change rolls back. Undetermined

Capture still reports "orders, key 42", because the statement ran. The committed-only re-read returns the old values, nothing changes, and no version is issued. The intent sees its values never arrived and ends "undetermined", shown to the user.

1. Vendor API → Source database: updates order 42
2. Source database → Change capture: statement completed
3. Source database: the transaction rolls back
4. Change capture → Store: hint: orders, key 42
5. Store → Source database: re-reads order 42, committed only
6. Store: unchanged: no new version

Outcome: *values never arrived: undetermined, shown to the user*

The sending worker crashes after the API call.

Another worker claims the intent. Its outcome is uncertain, so it checks the store before doing anything. Order 42 already shows the change, so the intent is judged on that, and the vendor is never called twice.

1. Worker A → Vendor API: sends the intent
2. Vendor API → Source database: updates order 42
3. Worker A: crashes before recording the send
4. Worker B: claims the intent: outcome uncertain
5. Worker B → Store: checks order 42 first
6. Store → Worker B: already holds the change

Outcome: *judged on what the store shows; the API is never called twice*

Someone edits order 42 in the vendor's own application at the same moment.

Their change arrives through capture like any other. If it set the same values, the intent can only say "state reached", because nothing proves which request caused it. If it set different values, the intent ends "undetermined" and the user sees the conflict.

1. Write queue → Vendor API: sends status = shipped
2. Vendor app user → Source database: edits order 42 directly
3. Vendor API → Source database: updates order 42
4. Change capture → Store: both changes captured and re-read
5. Store → User: the latest version of order 42

Outcome: *same values: state reached; different: undetermined*

A client was offline overnight.
It reconnects from its last version. If the outbox still holds everything after that version, it receives whole versions in order; if not, it resynchronizes its read models instead of replaying.

1. Client: comes back, last saw version 1001
2. Client → Store: resume after version 1001
3. Store → Client: outbox still holds it: 1002 onward, whole versions, in order
4. Store → Client: or, too old: resync the read models instead of replaying

Outcome: *resumed in order, or resynced; never a partial version*

Figure 2. Six scenarios, step by step: the ordinary path, a lost event, a rollback, a crashed sender, a concurrent edit in the vendor's application, and an offline client.

8 Related patterns

ORCA is assembled from known ideas; what it adds is how they're held together when the source is someone else's.

Change data capture tools (Debezium is the common example [7]) read the database's own log and treat what they read as the change. That needs access the vendor may not allow, and it trusts capture. ORCA needs neither: capture is a hint, and the source decides the truth.

Domain-driven design's anti-corruption layer [1] translates an outside model at the boundary of a service. ORCA makes it a library with no I/O, shared by the store and the client fallback, so the vendor's model has exactly one home.

CQRS [2,3] separates read models from the write side but assumes you own the write side. ORCA's belongs to the vendor; writes go through its API and are judged by what comes back. The transactional outbox [4] records changes with the data that produced them; ORCA adds versions in commit order, delivered whole.

Exactly-once processing [6] is replaced by honesty: one claimer, no blind resends, and three outcomes. Leases with fencing tokens [5] coordinate ORCA's own writers; they don't reach requests already in flight or people in the vendor's application.

Table 3. What ORCA takes from each pattern, and what it changes.

Pattern	What ORCA takes	What ORCA changes
Change data capture [6]	A stream of what changed, consumed from a position	Capture is only a hint; truth comes from a committed-only re-read, and gaps are expected and reconciled. No log access or schema change is needed.
Operational data store	A live, current copy for reads	A raw layer kept exactly as the source, so translations rebuild without touching the vendor
Anti-corruption layer [1]	Translate the outside model at the boundary	The translator is a library with no I/O, shared by the store and by clients' direct-to-source fallback, so the vendor's model has one home
CQRS read models [2,3]	Read-shaped data, separate from where writes go	Writes don't go to your store at all: they go through the vendor's API and are judged by what comes back
Transactional outbox [4]	Durable record of changes, written with the data	Versions are in commit order through a counter row, and delivered as whole groups so a consumer cannot skip part of one
Idempotent receivers, exactly-once [6]	Avoiding duplicate effects	Replaced by honesty: one claimer, never resend blind, and three outcomes (confirmed, state reached, undetermined)
Leases with fencing tokens [5]	Coordinating concurrent writers	Checked at enqueue and before send; explicitly not a guarantee against requests already in flight or people in the vendor's own application

9 When ORCA fits, and its limits

It fits when

- ✓ many tools depend on a system you do not control;
- ✓ reads vastly outnumber writes;
- ✓ the vendor's model is awkward;
- ✓ the vendor's API is incomplete;
- ✓ you cannot or will not alter the vendor's schema.

It doesn't fit

- × if you own the source (use its own change tracking and build read models directly);
- × if a few direct queries are enough;
- × if every write needs synchronous confirmation from the vendor in the user's path.

Assumptions

- Every table has a primary key, or is small enough to reconcile whole.
- The source supports a committed-only read.
- Capture is right often enough to be useful; reconciliation bounds the cost of it being wrong, not the delay.
- The vendor's API can make every write the platform needs.

Limits

- Freshness is bounded by capture's latency when it works, and by reconciliation's schedule when it doesn't.
- A checksum collision can hide a difference until the row changes again.
- Undetermined outcomes need a person.
- Conflicts with people in the vendor's application are detected, not prevented.

10 In practice

One implementation, against a vendor's practice-management system on SQL Server. It replaced direct reads from the office's tools; every app, dashboard and mobile tool reads from it, and almost every write goes through it. The stack is incidental. Any equivalent works: log-based capture such as Debezium, another database, Kafka for delivery, or SQLite on clients.

Capture

An Extended Events session on the database server, filtered to insert, update and delete, writes to a few bounded rolling files. A small service on the same server reads them and serves records over HTTPS as a long poll: a consumer asks for everything after its position and waits up to 25 seconds. Each response carries the next position and a gap flag; a consumer that falls behind the files gets the flag. Because it's taken at the server, it sees every writer: the vendor's desktop clients, its API, its web portal and its scheduled jobs.

A record names a table and the keys it has, or only an entity (an order), or nothing. An entity-only record re-reads that entity in every table keyed by it. A hint on a table whose key column isn't indexed is turned into a reconciliation of that table, so a hint never scans a large table.

Store

A Rust service keeps every source table in PostgreSQL, in its own schema: the same table names, columns and primary keys, plus one row checksum column. Types map directly, with two losses worth knowing: `datetime2(7)` keeps six fractional digits, and a NUL inside a string is dropped. One column of stored documents, nearly 200 GB, isn't copied; it's fetched from the vendor when someone opens a document.

Re-reads use `READCOMMITTEDLOCK`, so a rolled-back write is never copied. Range checksums are computed on both sides with dirty reads to keep them cheap. A summary that's wrong at worst copies a range for nothing or leaves it for the next pass; the copy itself is committed-only. A table whose key includes a `uniqueidentifier` is reconciled as one range, because the two engines order those differently. A table with no primary key is copied and checked whole.

The source never sees more than three connections: one for re-reads on hints, one for the first copy and reconciliation, and one for the nightly check. Every statement has a timeout. Every ten minutes the service compares the source's tables, columns, types and keys with what it copied; on a change it exits, a supervisor restarts it, and changed tables are copied again.

Reconciliation

Requests are numbered from a sequence. A table is covered up to the number that was current when its pass began, so a request that arrives during a pass gets another one. Large tables wait for an evening window; a pass still running when the window closes stops between ranges and resumes there the next night. Tables due go oldest attempt first, so one that keeps failing can't hold up the rest.

Read models and delivery

Shapes are built from the raw schema only, 500 entities at a time on a full build. Lookups are re-read every minute; a changed customer or contact rebuilds that customer's entities, and any other changed lookup rebuilds everything. Redis on a separate host fans versions out and keeps a stream per audience. Desktop clients keep open records plus the last year in LMDB, with a fuzzy index for search, and resume over a websocket by version; when the platform is down they run the same translation crate against SQL Server.

Writes

Each intent goes into an append-only, checksummed local log and shows on screen at once, with a five-second undo. A PostgreSQL queue claims intents with `FOR UPDATE SKIP LOCKED`. Leases take fencing tokens from a PostgreSQL sequence, and the vendor's own record locks are mirrored into the same leases. Mobile apps keep their own offline queue; the service that forwards their writes checks the record's lease first.

Table 4. Settings in this implementation.

Setting	Value
First-copy chunk	2,000 rows, 250 ms pause
Reconciliation range	20,000 rows
Statement timeout	60 s; 4 h for a whole-table read
Baseline reconciliation	every 24 h per table
Off-peak window	19:00 to 02:00
Reconciled at any hour	under 200,000 rows, or named by a hint
Schema check	every 10 min
Nightly check	200 random entities, re-checked after 30 s
Freshness logged	every 15 min, p50 and p95

11 Conclusion

ORCA treats the vendor's system as the truth and everything else as a signal. Capture says where to look; the source decides the truth. Reconciliation catches what capture misses, and writes go through the vendor and are judged by what comes back.

The combination is the point. A CDC pipeline usually assumes trustworthy capture and owned targets; CQRS usually assumes you own the write side. ORCA assumes neither, and says exactly which guarantees survive.

References

[1] Evans, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software.* Addison-Wesley, 2003.

[2] Fowler, M. "CQRS." martinfowler.com, 2011.

[3] Young, G. "CQRS Documents." 2010.

[4] Richardson, C. "Pattern: Transactional outbox." microservices.io.

[5] Kleppmann, M. "How to do distributed locking." martin.kleppmann.com, 2016.

[6] Kleppmann, M. *Designing Data-Intensive Applications.* O'Reilly, 2017.

[7] Debezium. "Debezium documentation." debezium.io.

Citation

Curtis, N. (2026). ORCA: Operational store, Read models, Change capture, Anti-corruption layer (Version 1.0). https://doi.org/10.5281/zenodo.23493408

```
@techreport{curtis2026orca,
  author  = {Curtis, Nathan},
  title   = {{ORCA}: Operational store, Read models, Change capture, Anti-corruption layer},
  year    = {2026},
  month   = oct,
  note    = {Version 1.0},
  doi     = {10.5281/zenodo.23493408},
  url     = {https://orcapattern.org}
}
```

